Advanced Computer Architecture

Chapter #4

Multithreaded Architecture

Ch#4 Multithreaded Architecture

Latency hiding techniques

Multithreading principles

Multithreaded architecture

Cluster computing

Neural Computing

Grid computing

Comparison of Cluster computing and Grid computing

Advantages and disadvantages of Grids

Multithreading

Multithreading

A technique that allows multiple threads within a single process to execute independently and share the resources of the process

Applied to a single processor to cause the single processor to switch between the contexts efficiently but at a time executes only one thread

But the multithreading in multiprocessors is each processor may execute independently, single thread of one process at the same time

The multiple processors may execute all threads of process concurrently and hence the overall processing will speed up

Basic principle behind multithreading is to hide the memory latency in scalable parallel systems by switching between threads

Latency Hiding Techniques

Latency Hiding Techniques

PE in multiprocessor system may need to access the remote (not local to PE) memory

Some time is required to fetch the data from remote memory ⇒ this time delay is called memory latency

This delay is due to delays of interconnection hardware, communication delay and memory response time

Even today, the processor is much faster than memory, so memory response is slow

⇒ There might be long latency in accessing the remote memory

Latency Hiding Techniques

Factors contributing to increase memory latency:

- Memory is not physically close to the processor
- The speed of memory is less compared to that of the processor
- Communication delay
- The speed of interconnection network between processor and memory

Handling the Latency Problem

To improve the performance of large scale computer systems, it is necessary to enhance their scalability and programmability

Programmability ⇒ the effective performance of the program depends upon many factors and memory latency is one of them

Some important mechanism to tackle the problem of latency are:

- Latency avoidance mechanism
- Latency reduction mechanism
- Latency hiding mechanism

Latency Avoidance Mechanism

Different techniques to avoid latency

Properly organize the user application ⇒ as to improve the localities ⇒ this technique requires that the parallel computing system should provide the supporting architecture such as cache coherency techniques, fast messaging technique

Another technique ⇒ user can explore the locality in the program by use of specific languages ⇒ the programmer can represent certain data structures in a way to improve the locality properties

Involve the compiler optimization to improve the locality

Latency Reduction Mechanism

The latency avoiding mechanism can be effective upto certain extent

At some critical point, it becomes difficult to enhance the data locality ⇒ necessity of reducing the latency

Latency can be reduced by using appropriate communication hardware and software

The scalable parallel system should use efficient communication hardware such as network interfaces and interconnection network

Latency Hiding Mechanism

Emphasizes on hiding the latency by the use of overlapping operations during latency

The operation of remote memory access is overlapped with another useful operation by the processor ⇒ processor need not wait till the remote data is accessed

Different techniques to hide latency are:

- Data prefetching techniques
- Use of coherent caches
- Use of relaxed memory consistency models
- Use of multithreading to do context switching to hide the latency

Data Prefetching Techniques

The required data for the program is to be prefetched and brought closer to the processor in advance

For this, gather information about such remote references in advance and start fetching that remote data

Move this prefetched data close to the processor before the data is actually needed

This technique finds out the expected data misses in the program, then accordingly prefetches that data and places it close to the processor before that data is really needed

Data Prefetching Techniques

Different types of prefetch techniques:

- Binding prefetch
- Non-binding prefetch

Binding Prefetch

There is a binding on the prefetch operation

When the required data is prefetched in advance then it must be loaded directly into the registers i.e., there should not be a time interval between prefetch and actual reference

Otherwise any other processor may modify the prefetched value before it is referenced by the processor which actually needs it

⇒ This indirectly decides the moment at which the binding prefetch can be issues

Non-binding Prefetch

The prefetch data is placed in the cache in advance before it is referred and it is maintained consistent by cache coherence mechanism

As the prefetched data maintains the consistency, there will not be any problem even if the processor reads the referenced value later

- The prefetch technique may be controlled by hardware or by software
- Thus, the prefetching techniques definitely hides the latencies due to remote access

Use of Coherent Caches

In large-scale computers, the caches are distributed and they are made coherent by implementing cache coherence protocols

Small-scale multiprocessor systems use snoopy protocol whereas large scale system use directory based protocol

Coherent caches help to hide latency by increasing the cache hit ratio for read operations

When the cache miss occurs, the number of cycles are wasted

The benefit of using distributed coherent caches is to reduce the cache miss

Use of Relaxed Memory Consistency Models

Different memory models developed for shared-memory multiprocessor systems

These memory models specify the ordering of global memory reads and memory writes generates by the same processor or different processors

The ordering of these reads/writes may cause the consistency problem

A memory consistency model indicates how the memory reads or writes generated by one processor should be observed by other processor in the multiprocessor system

There are sequential consistency and relaxed consistency memory models

Sequential consistency model:- The ordering of shared memory reads, writes and exchanges by all processors in system must be in a manner to maintain the program order in the individual processor

Relaxed memory model:- The sequential constraint of ordering is relaxed

Different relaxed memory models:

Weak consistency model,

Processor consistency model and

Release consistency model

Use of multithreading to do context switching to hide the latency

Multithreaded processor ⇒ multiple context processor

Multithreaded processor is designed to support multithreading

Has multiple sets context storage

When the processor executes a single thread, which requires a data from remote memory, then this processor does not have to wait

It simply switches to another thread and executes it

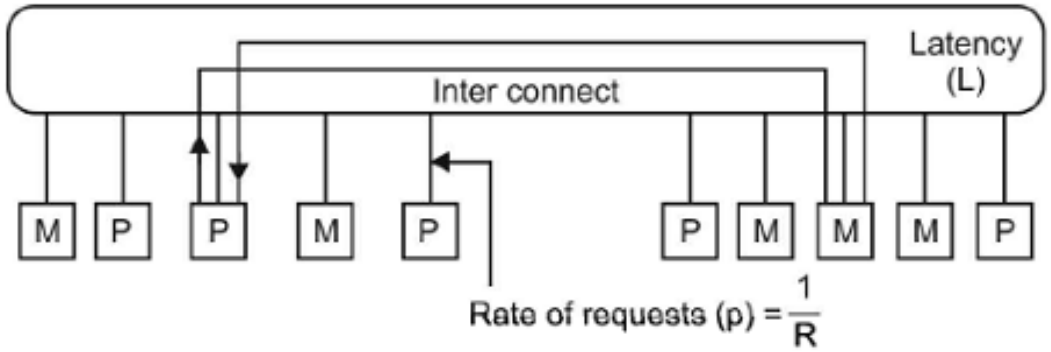
The memory access waiting period due to one thread is overlapped by the execution of another thread

Multithreading Principles

Multithreading Principles

The architecture environment of multithreading system is shown in figure

Distributed memories form a global address space



Multithreading Principles

To analyze the performance of the network, machine parameters are defined as:

Latency (L):

- Communication latency on a remote memory access
- The value of L includes the network delays, cache-miss penalty and delays caused by connections in split transactions

Number of threads (N):

- Number of threads that can be interleaved in each processor
- A thread is represented by a context, consisting of a program counter (PC), a register set and the required context status words

Multithreading Principles

Context switching overhead (C):

- Refers to the cycles lost in performing context switching in a processor
- This time depends on the switch mechanisms and the amount of processor states devoted to maintaining active threads

Interval between switches (R):

- Refers to the cycles between switches triggered by remote reference
- The inverse, P = 1/R is called the rate of requests for remote accesses
- This reflects a combination of program behavior and memory system design

Multithreading Principles

In order to increase efficiency

- One approach is to reduce the rate of requests by using distributed coherent caches
- Another approach is to eliminate processor waiting through multithreading

Issues of Multithreading & Solutions

There are some issues that are related with the overall performance in multithreading system, such as:

- Latency
- Number of threads and efficiency
- Context switching overhead
- Context switching policy

Latency

Memory latency ⇒ slows down the efficiency of the processor

Multithreading can be used to hide the memory latency

The operation are overlapped with memory latency and ideally the processor always remains busy ⇒ increasing the efficiency of processor

Number of threads and efficiency

A multithreaded processor is always kept busy by switching among the multiple contexts

Efficiency of the processor is given as:

$$\text{Efficiency} = \frac{\text{Processor busy time (or useful time)}}{\text{Processor busy time (or useful time)} + \text{Context Switching time} + \text{Processor idle time}} \quad \ldots(1)$$

Idle time is not always negligible due to many reasons (the threads/contexts may be blocked and processor becomes idle)

Number of threads and efficiency

Efficiency is dependent on the total number of threads in a process

If the number of threads is less, the processor may remain idle for some duration after completing switching among all available threads

If number of threads is very large, the processor continues switching among multiple contexts for long duration ⇒ in this case, the multithreaded processor continues context switching even after the memory latency is over

The processor goes into the saturation state of thread switching

In saturation state, the efficiency of the processor remains constant

Number of threads and efficiency

There is a threshold value for the total number contexts

Below threshold ⇒ efficiency of the processor is calculated using equ 1

Above threshold ⇒ saturation of the number of switches takes place and the efficiency is given by equ 2

$$\text{Efficiency} = \frac{\text{Processor busy time (or useful time)}}{\text{Processor busy time (or useful time)} + \text{Context Switching time}} \quad \ldots(2)$$

Context switching overhead

During the context switching some cycles are lost in saving the previous context and loading new context

This context switching overhead affects the efficiency of multithreaded processor

Large context switching overhead drastically slows down the efficiency

Practically, it is observed that switching overhead should not be more than 2 cycles for achieving good efficiency

Context switching policy

There are many policies for context switching but one has to consider the effect of policy or the efficiency of the processor

The context switching policy may affect the spatial locality in cache and pipeline dependencies ⇒ it may degrade the efficiency due to pipelining flushing

Solutions for effective multithreading:

- Selection of appropriate context switching policy
- Total number of threads should be at the threshold value
- Minimum context switching overhead

Multithreaded Architecture

Multithreaded Architecture

Three computational models (from the point of view of multithreaded architectures) are:

- Von-Neumann model
- Data flow model
- Hybrid Von-Neumann/data flow model

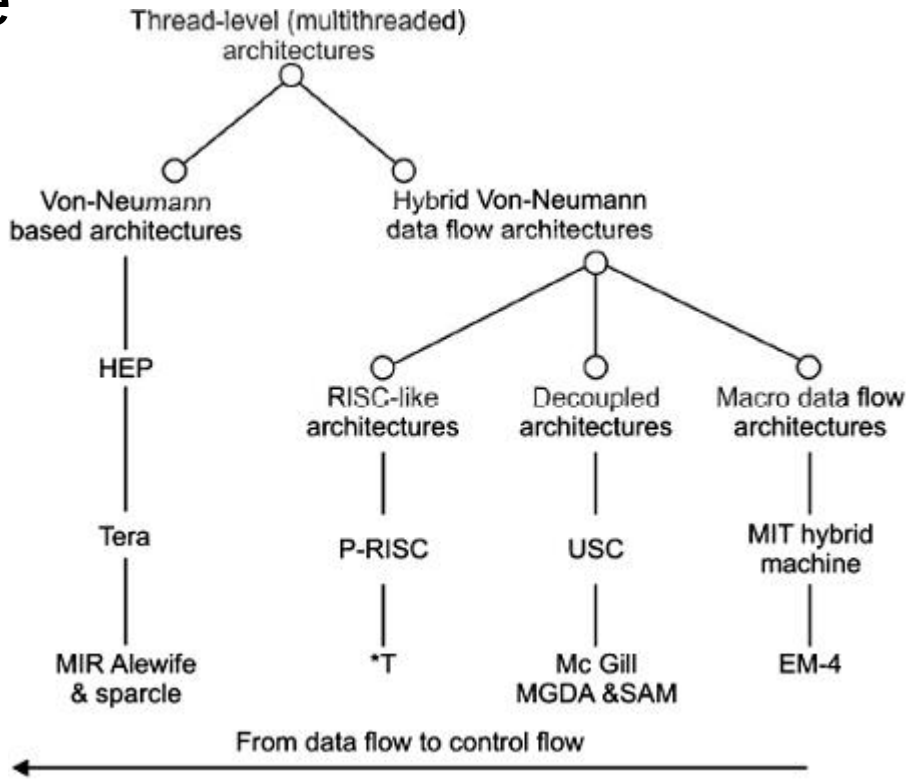


Fig: Classification of multithreaded architectures

Sequential flow control/Von-Neumann/Conventional model

The flow of control and data are separated

Instructions should be executed sequentially from top to bottom

Any deviation from this order should be explicitly defined by special control instructions like GOTO, JUMP, CALL etc

Hardware support for this sequential execution scheme consists of the program counter (PC) which is automatically incremented after executing ordinary instructions

Data flow model

In this model, the control is tied to the flow of data

The order of instructions in the program text has no effect on the execution order

An instruction is executed if all data needed for execution is available

As data can be available for several instructions at the same time, these instructions can be executed in parallel

Hybrid model

They combine features of both the data flow and control flow models

Hybrid multithreaded machines can be classified according to their position in the scale of computational models

Macro data flow architectures are the closest to the original data flow machines

Decoupled architectures are based on the control taken concept and present the next step towards control flow machines

RISC like hybrid architectures are the closest relatives of the original control flow machines

They work on a parallel control flow model based on parallel control operators

Cluster Computing

Cluster Computing

When two or more computers are used together to solve a problem ⇒ considered as a computer cluster

A collection of interconnected stand alone computers working together as a single, integrated computing resources

Consists of Nodes, Cluster middleware, Network, OS

Characteristics of Cluster Computing

Consists of many of the same or similar type of machines

Coupled using dedicated network connections

All machines share resources such as a common home directory

Must have software such as an MPI (message passing interface) implementation to allow programs to be run across all nodes

Why cluster?

High Availability (HA) / High Performance Computing i.e., getting higher reliability or for high performance to get greater Computing (HPC) that is used to get greater computing power than a single machine can provide

Cost, i.e., a cluster of the same size and power as a mainframe is many time cheaper than a mainframe

Types of Clusters

High Performance Clusters (HPC): Parallel, tightly coupled applications

High Throughput Cluster (HTC): Large number of independent tasks

High Availability Clusters (HA)

- Generally small, less than 8 nodes
- Multiple communication paths
- Used for mission critical applications

Load Balancing Clusters: Web servers, Mail servers

Hybrid Clusters: For multiple types at once such as HPC + HA

Pros of Cluster Computing

Clusters are price/performance computational engines

Mainstream tools for a variety of scientific fields

Expanded performance from HPC to:

- High availability
- Visualization

Benefits come due to:

- Using inexpensive commodity servers
- Open source software
- Large and expanding community of developers

Cons of Cluster Computing

Could be hard to manage $\Rightarrow$ need high experience

High-performance I/O $\Rightarrow$ still challenging

Technology is changing very rapidly $\Rightarrow$ scaling up issue

Largest problem in clusters is software skew i.e., when the software configuration on some nodes is different than on others, even small differences can cripple a parallel program

Neural Computing

Neural Computing

Neural computers are trained (not programmed) so that given a certain starting state (data input), they either classify the input data into one of the number of classes or cause the original data to evolve in such a way that a certain desirable property is optimized

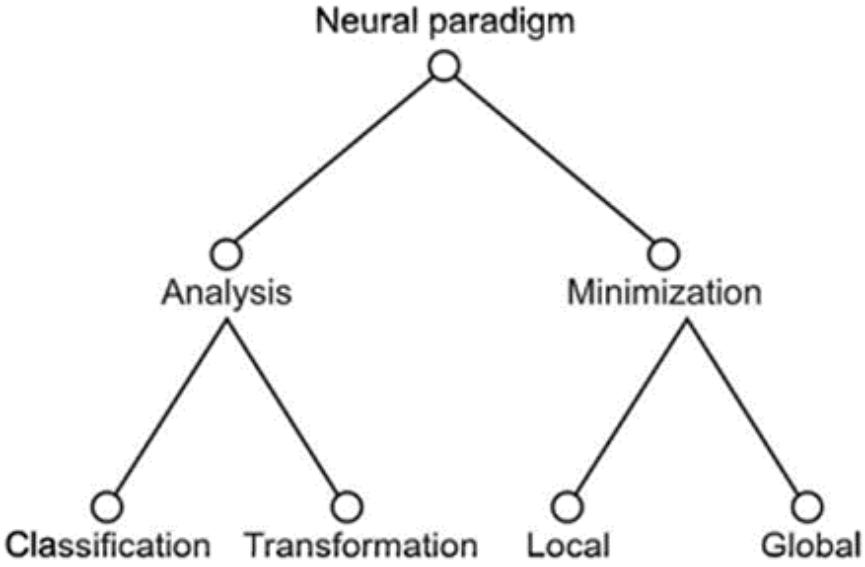


Fig. Neural Computing Paradigm

Classification - Involves many inputs and few outputs trained

Transformation - Involves many inputs and many outputs trained

Local - Involves many inputs and few outputs rule-based

Global - Involves many inputs and one output rule-based

Neural computers implement data parallelism and functional parallelism

Neural Network versus Conventional Computers

Conventional computers use algorithmic approach i.e., the computers follows a set of instructions in order to solve a problem

Neural networks process information in a similar way the human brain does. The network is composed of a large number of highly interconnected PEs (neurons) working in parallel to solve a specific problem

Neural networks learn by examples. They cannot be programmed to perform specific task

The neural networks and conventional algorithmic computers are not in competition but complement each other

Neural Network Software Packages

Few example cases - Left as an exercise

Grid Computing

Grid Computing

It is closely related to the cluster computing and is also called as **grid clusters**

Grids connect collection of computers which do not fully trust each other and hence operate more like a computing utility than like a single computer

Additionally grids typically support more heterogeneous collections than in clusters

Grids use the resources of many separate computers connected by a network (usually the Internet) to solve large-scale computation problems

Grids serve to manage the collection of jobs to computers which will perform the work independently of the rest of the grid cluster

Resources such as storage may be shared by all the nodes but intermediate results of one job do not affect other jobs in progress on other nodes of the grid

Features of Grid Computing

Classification of Grids

Computational Grids:- Includes CPU scavenging grids which focuses primarily on computationally intensive operations

Data Grids:- Performs the controlled sharing and management of large amounts of distributed data

Service/Equipment Grids:- They have a primary piece of equipment e.g., a telescope and where the surrounding grid is used to control the equipment remotely and to analyse the data produced

Comparison of Cluster Computing and Grid Computing

Advantages and Disadvantages of Grids

Advantages:

Gives substantial computing power for simulations, complex computations and analysis

The existing resources may be better exploited

It may offer scalability via incremental increase of resources

Disadvantages:

Indeterminate quality of service (QoS)

Rapidly changing infrastructure

Grid technologies are VARIED, some are IMMATURE, some require additional specialized high-capacity communication links